DLCD - Lab Project
Simulating a Pipelined MIPS Processor in gem5

Team:
Chinmay P N - PES1UG25CS145
Divya Dosetty - PES1UG25CS174
Diya Nekkarkad Kamath - PES1UG25CS173
Disha S - PES1UG25CS168

Introduction

- This project simulates a pipelined MIPS processor in gem5 and compares it with a simple, non-pipelined CPU.
- Aim: show how much pipelining speeds up programs, and what slows it down (hazards).

Pipeline Theory

A simple CPU finishes one instruction before starting the next. A pipelined CPU splits work into stages so several instructions are in progress at once, like an assembly line.
Classic 5 stages: fetch, decode, execute, memory, write-back.
Hazards: data (needs a result not ready yet), control (a branch changes what comes next), structural (two instructions need the same hardware).
Fixes: forwarding (pass a result early), stalling (wait), branch prediction and flushing (guess, and throw away wrong work).

Setup

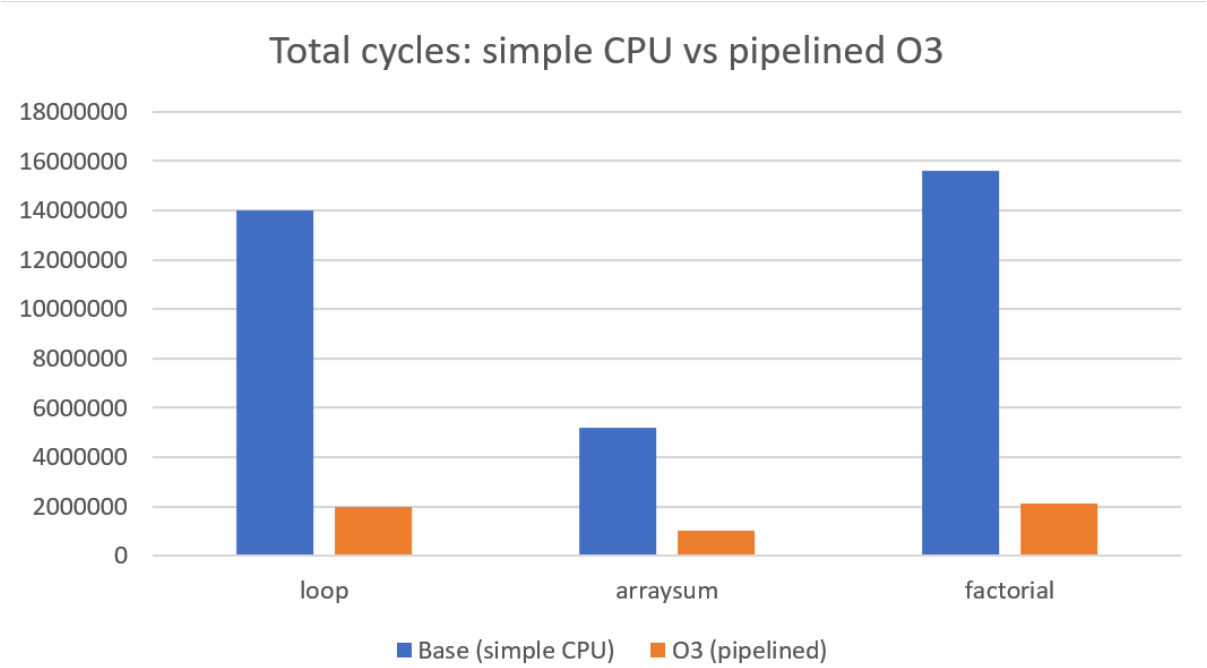
- gem5 built for the MIPS target, run in GitHub Codespaces (Ubuntu).
- Programs compiled with a MIPS cross-compiler (static, no C library).
- Simple CPU is TimingSimpleCPU. The pipelined CPU is MipsO3CPU, a wide out-of-order CPU, so part of the speedup comes from that and not from pipelining alone. MipsMinorCPU was tried first, but it stopped after 4 instructions in this gem5 version, so it was not used.
- Programs: loop (sums numbers from 1 to 1,000,000), arraysum (sums a 10,000-element array 50 times), factorial (factorial of 10, repeated 100,000 times).

Results: simple CPU vs pipelined CPU

Main table (compare cycles)

Program	Simple CPU cycles	Pipelined cycles	Speedup (simple/pipelined)
loop	14,000,560	2,000,593	7.00x
arraysum	5,168,492	1,037,671	4.98x
factorial	15,600,552	2,100,778	7.43x

Simple CPU vs Pipelined (O3) Bar Graph



Two independent baselines agreed: Chinmay's and Divya's simple CPU runs gave identical numbers.

IPC table

Program	Instructions (no nops)	Simple IPC	Pipelined IPC
loop	5,000,013	0.357	2.499
arraysum	2,040,215	0.395	1.966

factorial	5,700,010	0.365	2.713

The simple CPU's instruction count includes delay-slot nop instructions, which the pipelined model does not count. IPC is calculated from instructions committed excluding nops, which are identical on both CPUs.

A nop is a no-operation instruction. MIPS always runs the instruction after a branch (the branch delay slot), and the compiler fills it with a nop when nothing useful fits. Our loops run 1,000,000 times, so there are about 1,000,000 extra nops on the simple CPU.

Cache experiment

Program	Pipelined, normal cache (cycles)	Pipelined, 8 kB cache (cycles)
loop	2,000,593	2,000,593
arraysum	1,037,671	1,859,727
factorial	2,100,778	2,100,778

Hazard analysis

We ran three small MIPS programs on gem5 using MipsO3CPU, a pipelined CPU. Its stages are Fetch, Decode, Rename, Issue/Execute/Writeback and Commit. We tried the simpler MipsMinorCPU first, but it stopped after 4 instructions in this gem5 version, so we used MipsO3CPU. We recorded a pipeline trace (O3PipeView) and read the counters in stats.txt.

Because this CPU can finish more than one instruction per cycle, a CPI below 1 is normal.

Program	Instructions (no nops)	Cycles	CPI (cycles/ instructions)	Evidence
data_hazard	600,011	400,505	0.667	forwarding in the trace
control_hazard	2,150,116	2,025,285	0.942	50,147 branch mispredicts
structural_hazard	1,239,856	1,036,754	0.836	load/store queue full 722 times

- control_hazard has the highest CPI (0.942) - branch heavy

Results

- The pipelined CPU finished the same three programs in far fewer cycles than the simple CPU: about 7.0 times fewer for loop, 5.0 times for arraysum and 7.4 times for factorial.
- Arraysum gained the least, because it spends more time waiting for memory.
- With the data cache reduced to 8 kB, arraysum needed about 1.8 times more cycles, while loop and factorial did not change, because they use almost no memory.

Conclusion

- Pipelining speeds up programs by overlapping instructions. In our simulation the pipelined CPU was 5 to 7 times faster than the simple CPU.
- Part of the speedup comes from the pipelined CPU being out-of-order, not from pipelining alone.
- Hazards (data, control and structural) cause stalls and flushes that reduce the gain, and branch-heavy code costs the most. Cache size also changes performance, especially for memory-heavy programs.